Course Discussions

Solution: Jewels and Stones (easy)

Problem Statement

Given two strings. The first string represents types of jewels, where each character is a unique type of jewel. The second string represents stones you have, where each character represents a type of stone. Determine how many of the stones you have are also jewels.

Examples:

1. **Example 1**:
   - Input: Jewels = "abc", Stones = "aabbcc"
   - Expected Output: 6
   - Justification: All the stones are jewels.
2. **Example 2**:
   - Input: Jewels = "aA", Stones = "aAaZz"
   - Expected Output: 3
   - Justification: There are 2 'a' and 1 'A' stones that are jewels.
3. **Example 3**:
   - Input: Jewels = "zZ", Stones = "zZzZzZ"
   - Expected Output: 6
   - Justification: All the stones are jewels.

Solution

To solve this problem, we can use a HashSet for its property of constant-time lookups. First, we add each character (jewel) from the first string to a HashSet. This set now represents all unique jewels. Next, we iterate through each character (stone) in the second string and check if it is present in the HashSet. If it is, it means this stone is also a jewel. We keep a count of such stones. The final count gives us the total number of stones that are also jewels. The efficiency of this solution lies in the HashSet's ability to quickly check for the presence of a jewel, making the overall process highly efficient for large strings.

1. **Populate Jewel Set**: Traverse the jewels string and populate a hashset with each character.
2. **Count Jewels**: Traverse the stones string. For each character, check if it exists in the hashset. If it does, increment a counter.
3. **Return Result**: After traversing the entire stones string, return the counter.

Using a hashset allows for efficient storage and retrieval of jewel types, ensuring that we can determine the number of jewels in linear time.

Algorithm Walkthrough:

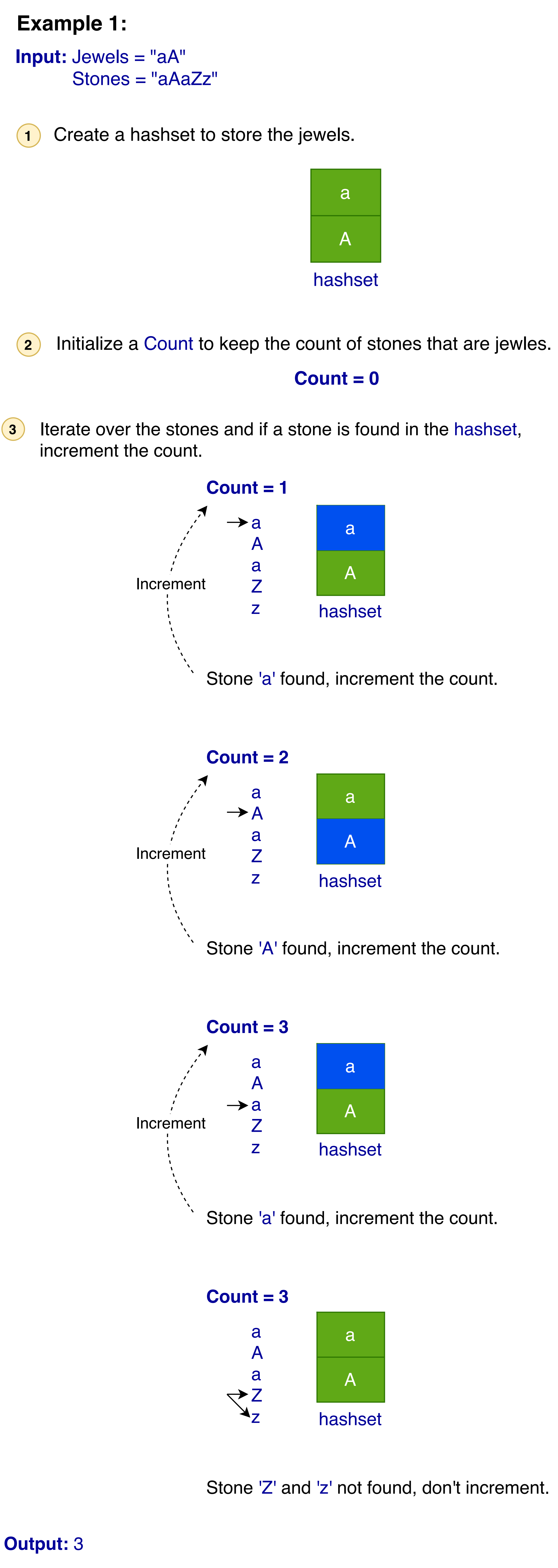
Given the jewels "aA" and the stones "aAaZz":

- Initialize an empty hashset and a counter set to 0.
- Traverse the jewels "aA" and populate the hashset: {'a', 'A'}.
- Traverse the stones "aAaZz". For each character:
  - Check if it exists in the hashset.
  - If it does, increment the counter.
- After traversing all the stones, the counter will be 3, which is the result.

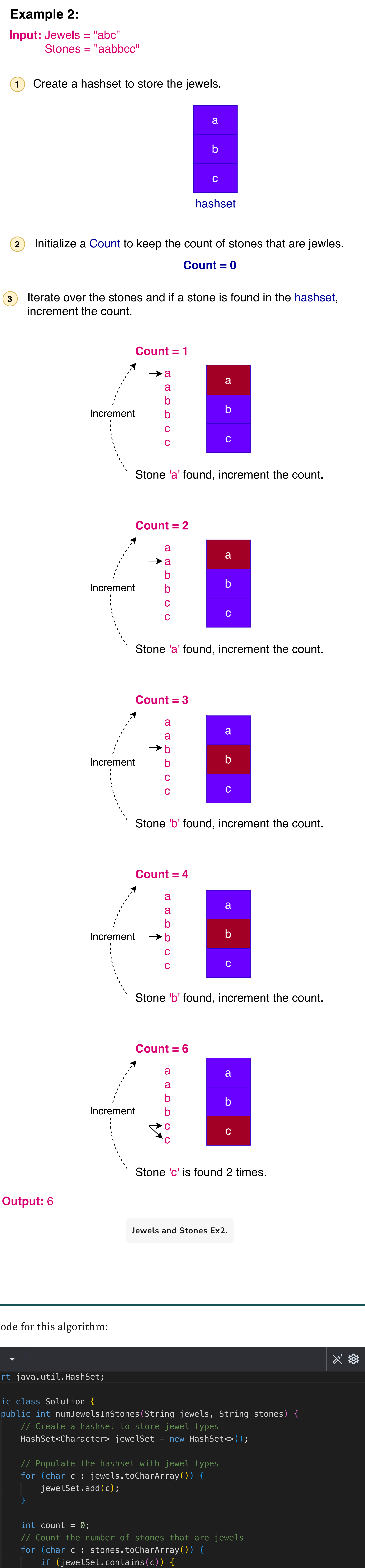
For the third example, with jewels "zZ" and stones "zZzZzZ":

- Initialize an empty hashset and a counter set to 0.
- Traverse the jewels "zZ" and populate the hashset: {'z', 'Z'}.
- Traverse the stones "zZzZzZ". For each character:
  - Check if it exists in the hashset.
  - Since all the stones are jewels in this case, the counter will increment for each stone.
- After traversing all the stones, the counter will be 6, which is the result.

Here is the visual representation of the algorithm:


Example 1:
Input: Jewels = "aA"
Stones = "aAaZz"
1 Create a hashset to store the jewels.
a
A
hashset
2 Initialize a Count to keep the count of stones that are jewles.
Count = 0
3 Iterate over the stones and if a stone is found in the hashset, increment the count.
Count = 1
Increment
a A a Z z
hashset
Stone 'a' found, increment the count.
Count = 2
Stone 'A' found, increment the count.
Count = 3
Stone 'a' found, increment the count.
Count = 3
Stone 'Z' and 'z' not found, don't increment.
Output: 3

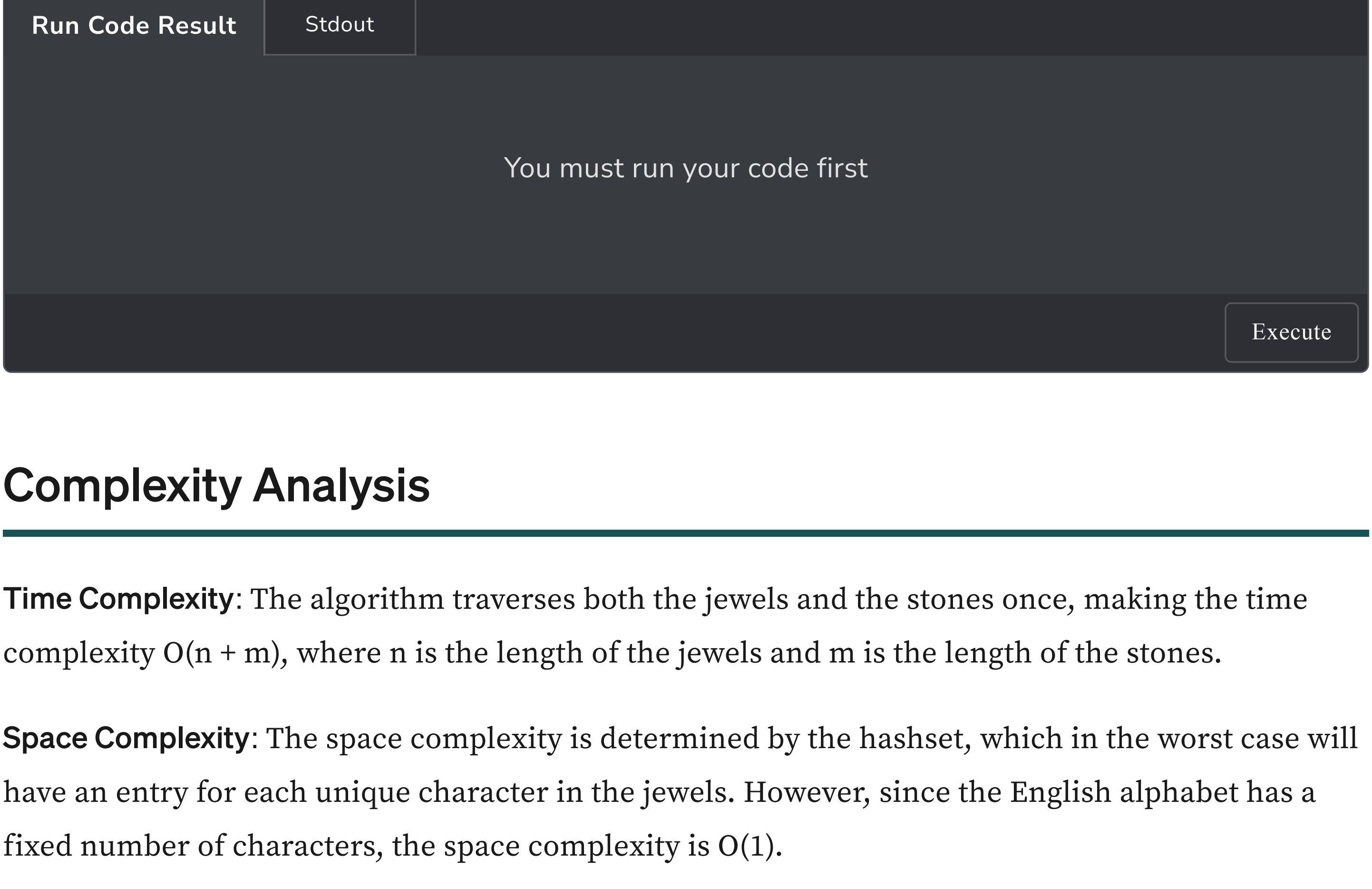
Jewels and Stones Ex1.


Example 2:
Input: Jewels = "abc"
Stones = "aabbcc"
1 Create a hashset to store the jewels.
a
b
c
hashset
2 Initialize a Count to keep the count of stones that are jewles.
Count = 0
3 Iterate over the stones and if a stone is found in the hashset, increment the count.
Count = 1
Increment
a a b b c c
Stone 'a' found, increment the count.
Count = 2
Stone 'a' found, increment the count.
Count = 3
Stone 'b' found, increment the count.
Count = 4
Stone 'b' found, increment the count.
Count = 6
Stone 'c' is found 2 times.
Output: 6

Jewels and Stones Ex2.

Code

Here is the code for this algorithm:


Java
Run Code Result
Stdout
Execute

```java
import java.util.HashSet;

public class Solution {
    public int numJewelsInStones(String jewels, String stones) {
        // Create a hashset to store jewel types
        HashSet<Character> jewelSet = new HashSet<>();

        // Populate the hashset with jewel types
        for (char c : jewels.toCharArray()) {
            jewelSet.add(c);
        }

        int count = 0;
        // Count the number of stones that are jewels
        for (char c : stones.toCharArray()) {
            if (jewelSet.contains(c)) {
                count++;
            }
        }

        return count;
```

You must run your code first

Complexity Analysis

Time Complexity: The algorithm traverses both the jewels and the stones once, making the time complexity O(n + m), where n is the length of the jewels and m is the length of the stones.

Space Complexity: The space complexity is determined by the hashset, which in the worst case will have an entry for each unique character in the jewels. However, since the English alphabet has a fixed number of characters, the space complexity is O(1).

← Previous
Problem 2: Jewels And Stones (Easy)

Next →
Problem 3: Unique Number Of Occurrences (Easy)

☐ Mark as Completed